

SPM Report

Master's in CS, Artificial Intelligence

Gabriele Di Giulio

Contents

1	Introduction and Objectives	2
2	Implementations	2
2.1	C++ Threads	2
2.2	Task-based OpenMP	3
2.3	MPI + OpenMP	3
3	Correctness and Validation	4
3.1	Correctness Results	4
4	Experimental Setup	5
4.1	MPI Sweep	5
4.2	Task Granularity	5
4.3	Decoupling Norm Granularity from SpMV Granularity	7
4.4	Sparsity Structure Analysis	8
4.5	Strong Scalability Analysis	9
4.6	Weak Scalability Analysis	10
4.6.1	Results: Proportional Growth	10
4.6.2	Results: Constant Vector Size Scaling	11
5	Task-Based vs Work-Sharing Scheduling	13
5.1	Work-Sharing OpenMP	13
5.2	MPI + OpenMP Work-Sharing	13
5.3	Performance Evaluation: Task vs Work-Sharing	13
6	Conclusions	14

1 Introduction and Objectives

This project focuses on the iterative sparse matrix-vector computation applied to evolving matrices. Specifically, given a sparse matrix $A \in \mathbb{R}^{N \times N}$ and an initial vector x_0 , the algorithm repeatedly applies an SpMV product followed by a normalization phase. Every fixed number of epochs, the rows of the matrix undergo a circular shift. Throughout the execution, the matrix remains sparse, its dimensions do not change, and the total number of non-zero elements remains constant.

The goal of the project is the development and performance evaluation of three main parallel implementations, derived from the sequential reference code `iterative_SpMV.cpp`:

1. **C++ Threads:** Shared-memory parallelization based on a Thread Pool.
2. **OpenMP:** Shared-memory parallelization leveraging high-level directives.
3. **MPI + OpenMP:** Hybrid and distributed implementation for execution on multi-node clusters.

For the latter two, both built on OpenMP, work is expressed either as task-based (`#pragma omp task`, used in the main analysis) or work-sharing (`#pragma omp for`, examined separately in Section 5).

The objective is not merely the implementation of a parallel scheme, but rather the systematic analysis of the impact that factors such as sparsity, load balancing, granularity, and communication overhead exert on computational efficiency. This performance investigation constitutes the true core of the study.

2 Implementations

2.1 C++ Threads

The C++ thread-based implementation relies on a pool of persistent workers, instantiated only once at program startup and kept active for the entire execution. The system follows a single-producer/multiple-consumer model, in which the main thread acts as the sole producer, enqueueing work that the persistent worker pool consumes. This approach eliminates the massive overhead that would result from continuously creating and destroying threads at every single iteration of a loop. The main data structures, such as the vectors in memory, are allocated outside the main loop and passed to the workers by reference. In this way, threads do not receive unnecessary copies of the data, but only the coordinates (a start and end range) of the memory portion they need to process.

To streamline operations, the thread pool exposes a single generic `enqueue` method built on `std::packaged_task` and `std::future`. Regardless of whether the wrapped callable returns a value (e.g., the partial sums computed during the dot product) or nothing (e.g., vector normalization), it is uniformly reduced to a zero-argument, `void`-returning task before being pushed onto the internal dispatch queue; the actual result, if any, is retrieved separately through the associated future. This uniform interface greatly simplifies the architecture. Furthermore, the system inherently provides dynamic load balancing through pull-based scheduling. Work is statically partitioned into chunks of contiguous rows, and threads autonomously pull the next available chunk from the central queue as soon as they finish their current one. This mechanism helps mitigate the effects of highly unbalanced (*Irregular*) matrices: if a thread is assigned a chunk containing an unusually dense row and takes longer to process it, the remaining chunks are naturally picked up by threads that finish earlier, absorbing part of the imbalance without needing to recompute a fixed work assignment at every matrix shift.

Finally, synchronization is seamlessly integrated into this workflow. The main thread assigns the work and then waits for task completion by collecting the "futures" (completion signals or partial results). This wait acts as a natural synchronization barrier, ensuring that the swapping of vectors for the next iteration occurs only after all worker threads have completed their portion of the work, thus guaranteeing the correctness of the iterative computation.

2.2 Task-based OpenMP

The OpenMP implementation follows this same single-producer/multiple-consumer scheme, but is built entirely with compiler directives instead of an explicit thread pool. A single `#pragma omp parallel` region is opened only once at the beginning and remains active throughout the execution. Within it, a `#pragma omp single` block delegates control of the iterative flow to a single thread in the team, while all others wait, ready to fetch work as soon as it is generated.

Work generation and scoping are expressed via `#pragma omp task` directives, with matrix rows divided into blocks of `chunk_size`. Each task copies its own small state data (`start`, `end`, and `row_shift`) via `firstprivate`, while larger data structures like the matrix and vectors remain `shared` and are accessed by reference, exactly as in the C++ pool. Dynamic load balancing is handled entirely by the OpenMP runtime, which assigns tasks to free threads as they become available. As soon as a thread finishes a task, it immediately fetches another one from the queue, naturally absorbing part of the imbalance caused by dense rows falling within a chunk of the *Irregular* matrix. Furthermore, partial sums, such as those needed for the dot product in `dot_omp_tasks`, are automatically combined using `taskgroup task_reduction` and `in_reduction`. The runtime maintains a private accumulator for each task and merges them autonomously, replicating at the compiler level the same sequential collection logic seen in the C++ version. Ultimately, phase synchronization is expressed with `#pragma omp taskwait` (or by closing the scope of a `taskgroup`), ensuring that every task of a given kernel has completely finished writing to memory before the next step begins, such as swapping the vectors.

2.3 MPI + OpenMP

The hybrid implementation combines MPI for inter-node communication and OpenMP tasks for intra-node parallelism. The environment is initialized with `MPI_THREAD_FUNNELED`: all network calls (`MPI_Allreduce`, `MPI_Allgatherv`, `MPI_Bcast`) occur from the single thread executing the `#pragma omp single` block, ensuring the correctness of the coexistence between MPI and OpenMP tasks.

During the initial setup, an nnz-balanced physical partitioning is established: rank 0 divides the matrix only once using `partition_rows_by_nnz`, assigning each rank a fixed physical row interval with an approximately equal number of non-zero elements. Matrix evolution, specifically the circular row shift, is handled entirely as a logical permutation in memory: once every rank holds the fully assembled physical result vector (after the gathering phase), a parallel permutation step writes each physical value p to its logical destination $(p + \text{row_shift}) \bmod n$, at zero network cost.

The `spmv_local_omp_tasks` kernel computes partial results on each rank's assigned physical rows; these are then assembled into the global vector via a single `MPI_Allgatherv` call, using a communication plan (counts/displacements) derived once from the initial physical partitioning and reused unchanged across all iterations.

This design also employs two-level reductions: vector norms are calculated locally first on the memory-resident portion using OpenMP tasks, and the partial result is subsequently combined globally with `MPI_Allreduce`, which transmits a single `double` per rank.

Finally, hybrid flow management is orchestrated by encapsulating the entire iterative cycle within a persistent `#pragma omp parallel` region. The thread delegated by the `single` block

takes full responsibility for generating the local tasks, waiting for their completion via `#pragma omp taskwait` or `taskgroup`, and invoking all MPI collective functions.

3 Correctness and Validation

The correctness of the three parallel and distributed implementations was verified through an exhaustive cross-validation. The outputs were validated by systematically comparing all versions against each other, including the sequential baseline. The testing strategy was conducted on two problem scales:

- **Small-scale problem** ($N = 1000$, $NZ = 250\,000$): Cross-validation was performed by comparing the final Rayleigh value, the hexadecimal checksum of the resulting vector, and an element-wise comparison of the full vector dump files against the sequential baseline, using the same numerical tolerance of 10^{-12} .
- **Large-scale problem** ($N = 1\,000\,000$, $NZ = 250\,000\,000$): The validation approach was restricted exclusively to the final Rayleigh value, acknowledging the expected divergence of the checksum.

The decision to exclude the vector dump and checksum for large-scale matrices is driven by intrinsic technical limitations. First, exporting and comparing vectors with millions of elements introduces prohibitive storage and I/O bottlenecks. Second, parallel execution inherently changes the order of mathematical accumulations; due to floating-point non-associativity, this non-deterministic reordering alters the least significant bits, rendering strict bitwise checksums unusable. A non-matching checksum is fully expected in parallel environments—particularly with MPI collectives like `MPI_Allreduce` and `MPI_Allgatherv`—even when the underlying values remain numerically equivalent. Consequently, rather than demanding bitwise equality, validation relies on a predefined numerical tolerance of 10^{-12} . The parallel computation is considered correct if the absolute difference between its final Rayleigh value and the sequential baseline falls within this threshold.

3.1 Correctness Results

The validation tests were executed using **16 threads** for the shared-memory C++ Threads and OpenMP implementations. For the hybrid MPI+OpenMP implementation, the execution was configured based on the findings of the architectural sweep: distributed across **8 physical nodes** using **1 MPI process per node** (8 total ranks) and **16 OpenMP threads per rank**.

Table 1 reports the cross-validation results for the small-scale problem, where it was also possible to compare the identity of the dump files.

Version	Rayleigh	Checksum	Dump Match (tol. 10^{-12})
Sequential	11.262493...	0xc149817131f392ed	Yes
C++ Threads	11.262493...	0xc149817131f392ed	Yes
OpenMP Tasks	11.262493...	0xc149817131f392ed	Yes
MPI+OpenMP	11.262493...	0xe92449eb89b31c47	Yes

Table 1: Cross-validation results for the small-scale problem ($N = 1000$, $NZ = 250\,000$).

Table 2 shows the results for the large-scale problem. As anticipated, observing the checksum confirms the bitwise divergence among the parallel runs, whereas algorithmic correctness is validated by the stability of the Rayleigh value within the predefined tolerance.

Version	Rayleigh	Checksum
Sequential	10.693716...	0xad8f57527b104753
C++ Threads	10.693716...	0x7a6e0640844bbf11
OpenMP Tasks	10.693716...	0xb7180374b2abc6cd
MPI+OpenMP	10.693716...	0xeb1c3d55514442be

Table 2: Cross-validation results for the large-scale problem ($N = 1\,000\,000$, $NZ = 250\,000\,000$).

4 Experimental Setup

4.1 MPI Sweep

In hybrid MPI+OpenMP architectures, performance depends on the balance between network parallelism (MPI ranks) and intra-node parallelism (OpenMP threads). For a memory-bound kernel like SpMV, increasing MPI ranks intensifies the network traffic of the `MPI_Allgather` collective, while shifting load to OpenMP threads moves the cost to shared-memory synchronization within the node. To identify the optimal balance under fixed hardware resources, a sweep was conducted on 8 nodes (16 physical cores each, 128 total workers), varying OpenMP threads per rank (1, 2, 4, 8, 16), plus an oversubscribed case at 32 threads. Tests used an *Irregular* matrix ($N = 1\,000\,000$, $NZ = 250\,000\,000$), SpMV granularity fixed at 1024, averaged over 3 repetitions per configuration.

Ranks	Threads	Total (s)	Comp (s)	Comm (s)	Red (s)	Scatt (s)
128	1	34.972	20.819	14.413	6.603	6.556
64	2	36.830	22.768	14.171	14.838	4.021
32	4	30.998	15.871	16.687	13.913	1.887
16	8	31.020	13.949	19.811	8.535	1.015
8	16	30.077	17.965	11.796	11.188	0.551
8	32	31.258	18.807	14.149	6.788	5.639

Table 3: Median total time and per-phase peak times of the hybrid sweep on 8 nodes varying the MPI/OpenMP balance (SPMV Chunk = 1024)

Three observations emerge from these results. First, the configuration with 8 ranks and 16 threads produces the minimum total time (30.08 s) and is adopted as the fixed topology for subsequent experiments; it should be noted, however, that this configuration, along with 32 ranks/4 threads (31.00 s) and 16 ranks/8 threads (31.02 s), falls within a narrow band, meaning any of the three would have been a reasonably valid choice. The extreme rank counts (128 and 64 ranks) are instead clearly worse, at 34.97 s and 36.83 s respectively. Second, the degradation across configurations appears non-monotonic: total time worsens moving from 128 to 64 ranks (34.97 s to 36.83 s) despite near-identical communication cost, coinciding instead with a rise in the reduction phase to 14.84 s at 64 ranks, the highest value recorded in the sweep. Communication time at 16 ranks is also comparatively high, with `MPI_Allgather` reaching 19.81 s, the highest value among all configurations tested. Finally, oversubscription at 32 threads is associated with a modest performance drop relative to 16 threads (31.26 s vs 30.08 s); local computation time increases only slightly (18.81 s vs 17.97 s), while scatter time increases roughly tenfold (0.55 s to 5.64 s), which may point to the permutation step being more sensitive to core oversubscription than the SpMV kernel at this problem size.

4.2 Task Granularity

Task granularity is controlled by `SPMV_Chunk`, defining the number of matrix rows assigned to each OpenMP task or Thread Pool job. Here, this value was applied to both the SpMV kernel

and vector operations (`norm_chunk_size = SPMV_Chunk`), so the measured total time reflects a single granularity across the entire computational cycle. This coupling is removed in the next section, where the vector-ops chunk varies independently. The parameter was varied in powers of two from 256 to 16384 on the large-scale matrix ($N = 1\,000\,000$, $NZ = 250\,000\,000$, *Irregular* mode), comparing all three implementations at 16 and 32 threads; for MPI+OpenMP, the winning topology (8 nodes, 1 rank/node) was kept fixed. Median results over 3 repetitions are in Table 4.

Table 4: Median total execution times as the task granularity varies, with SPMV Chunk = Norm Chunk Size.

Implementation	Num. Threads	SPMV Chunk	Time (s)
C++ Threads	16	256	89.84
		512	84.69
		1024	82.89
		2048	81.62
		4096	81.98
		8192	82.74
		16384	101.81
	32	256	62.05
		512	55.84
		1024	51.83
		2048	51.38
		4096	51.63
		8192	70.67
		16384	99.28
OpenMP Tasks	16	256	93.21
		512	86.55
		1024	83.75
		2048	82.68
		4096	82.43
		8192	83.68
		16384	120.18
	32	256	94.25
		512	70.24
		1024	60.12
		2048	54.60
		4096	54.01
		8192	81.56
		16384	125.77
MPI+OMP	16	256	33.68
		512	30.04
		1024	29.58
		2048	31.69
		4096	42.65
		8192	46.88
		16384	83.85
	32	256	64.12
		512	40.02
		1024	31.65
		2048	36.69
		4096	45.53
		8192	56.02
		16384	93.87

Total time is shaped by two competing costs that move in opposite directions as the chunk grows. At the lower extreme, fine-grained overhead (a chunk size of 256) causes scheduling and synchronization to dominate over useful work; this effect is amplified at 32 threads by heavier contention on the shared task queue, with OpenMP reaching 94.25 s. As chunk size grows, this overhead recedes and each implementation settles into its own sweet spot: the single-node versions land between 2048 and 4096 rows, while MPI+OMP performs best around 1024 rows

(29.58 s), closely followed by 512 (30.04 s). Past this point the balance tips the other way, though with architectural differences. For the shared-memory implementations, at ≥ 8192 rows, the sharp increase in execution time suggests that the working set begins to exceed local cache capacity, forcing reliance on slower main memory. The threshold for this degradation depends on active threads (starting earlier at 8192 for 32 threads than at 16384 for 16 threads), mirroring expected cache dynamics as doubling threads accelerates capacity bottlenecks. For the distributed MPI+OMP setup, the degradation threshold occurs earlier (starting already at 4096 rows), consistent with the heavier local memory bandwidth contention that arises when active threads compete over a reduced per-rank working set, independent of thread oversubscription. Using 32 threads produces an opposite effect across architectures: for C++ and OMP it lowers times in the sweet spot, partially mitigating bottlenecks, whereas for MPI+OMP it worsens them systematically. With the same number of ranks (8) and unchanged network traffic, this is consistent with local memory-bandwidth contention rather than a network effect. Since each rank already works on a reduced working set (1/8 of the matrix), the bandwidth margin exploitable by additional threads is smaller than in the single-node case, making oversubscription proportionally more detrimental.

4.3 Decoupling Norm Granularity from SpMV Granularity

The previous sweep showed the SpMV kernel favors a fine-to-medium granularity (`SPMV_Chunk` ≈ 1024) to cope with matrix irregularity. However, the interleaved vector operations (L_2 norm, normalization) have a linear access pattern and perfectly balanced workload: reusing the same fine granularity introduces unnecessary scheduling overhead. To isolate this effect, `SPMV_Chunk` was fixed at 1024 and `Norm_Chunk_Size` varied exponentially from 64 to 262144. All three implementations used 16 threads—the optimal topology for MPI+OMP, though not necessarily the peak configuration for single-node versions, where 32 threads previously proved faster. Median results are in Table 5.

Implementation	Norm Chunk Size	Vector Ops Time (s)	Total Time (s)
C++ Threads	64	37.13	117.10
	256	9.44	89.52
	1024	2.53	82.95
	4096	0.74	80.69
	16384	0.25	79.99
	65536	0.22	80.43
	262144	0.41	80.66
OpenMP Tasks	64	56.21	136.94
	256	13.35	93.10
	1024	3.40	83.41
	4096	0.85	83.97
	16384	0.21	80.36
	65536	0.20	81.58
	262144	0.40	80.81
MPI+OMP	64	31.47	48.62
	256	8.52	33.85
	1024	2.13	28.22
	4096	0.46	28.89
	16384	0.16	27.64
	65536	0.17	28.53
	262144	0.41	29.49

Table 5: Median vector-ops and total times as `-nc` varies (SpMV Chunk fixed at 1024, 16 threads).

The vector-ops time collapses by two orders of magnitude between the extremes, but not

monotonically. Below 256, scheduler overhead per micro-task dominates: at 64 rows, the cost of dispatching micro-tasks completely eclipses useful computation (56.21 s). The overhead vanishes almost linearly up to 16384, the point where task count and thread count are finally in balance, and vector-ops drops to 0.16 s for MPI+OMP, yielding the best total time overall (27.64 s). Push further, to 262144, and the logic reverses: only 4 tasks exist for 16 threads, so 12 of them sit idle while the remaining work serializes, and the time climbs back up.

Decoupling the two granularity parameters is therefore essential: the sparse kernel needs fine blocks ($-c \approx 1024$) to balance the irregular non-zero distribution, while the dense vector operations need coarse blocks ($-nc \approx 16384$) to maximize memory bandwidth and suppress synchronization overhead.

4.4 Sparsity Structure Analysis

A critical aspect of SpMV performance is the memory access pattern induced by the distribution of column indices. When multiplying a sparse matrix by a dense vector x , this distribution determines how the CPU reads x . To quantify the impact of the memory wall and cache hierarchy, the algorithm was tested under two opposite structural patterns on a matrix with $N = 1\,000\,000$ and $NZ = 250\,000\,000$:

- **Regular Mode:** Non-zeros are adjacent, favoring sequential access to x and maximizing hardware prefetcher effectiveness.
- **Irregular Mode:** Non-zeros are pseudo-randomly distributed, forcing unpredictable access to x and sharply increasing cache misses.

Tests compared single-node execution (C++ Threads and OpenMP Tasks, 16 threads) against the distributed setup (MPI+OMP, 8 nodes). Results are reported in Table 6 and Figure 1.

Implementation	Reg. Total	Reg. Comp	Irr. Total	Irr. Comp	Δ Total
C++ Threads	82.09	82.08	80.34	80.33	-2.1%
OpenMP Tasks	80.81	80.80	80.38	80.37	-0.5%
MPI+OMP	17.45	11.56	29.25	17.65	+67.6%

Table 6: Total and local computation execution times (in seconds) by non-zero element distribution.

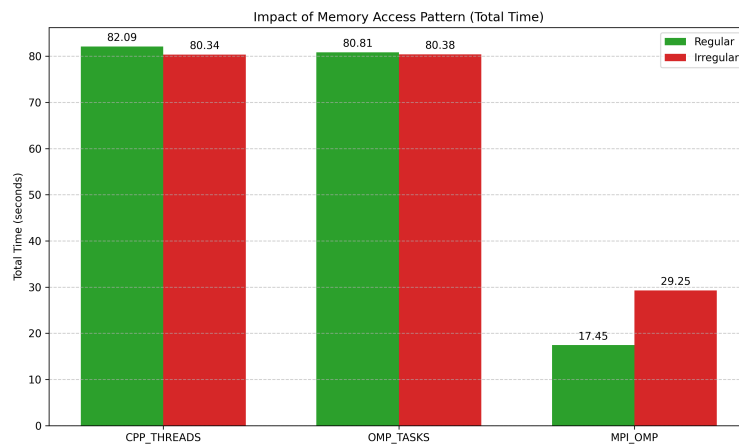


Figure 1: Impact of memory access pattern across implementations. Note the sharp degradation in the distributed approach.

On shared-memory architectures, the system demonstrates remarkable single-node resilience. In these setups, the irregular pattern induces no meaningful performance penalty. Execution time differences remain within a negligible 0.5% to 2.1% margin (where the irregular mode even registers slightly faster times), falling entirely within statistical measurement noise. Although random access inherently increases local cache misses, the node’s robust memory bandwidth, combined with 16-way multithreading, fully masks the added latency.

In contrast, the distributed MPI+OMP approach across 8 nodes experiences a severe performance collapse, effectively hitting a memory wall. When switching to the irregular pattern, the total execution time jumps dramatically from 17.45 s to 29.25 s, resulting in a 67.6% slowdown. This massive degradation is heavily driven by the pure local compute time (as detailed in Table 6), which alone increases by approximately 52% (from 11.56 s to 17.65 s).

4.5 Strong Scalability Analysis

The next step in the performance analysis consists of evaluating the *Strong Scalability* of the distributed implementation: the global problem size ($N = 1\,000\,000$, $NZ = 250\,000\,000$) remains strictly fixed while the number of physical nodes increases from 1 to 8. The optimal configuration identified in the previous sections was used: 1 MPI rank per node, 16 OpenMP threads per rank, SPMV_Chunk=1024, and Norm_Chunk_Size=16384.

Two distinct metrics were used:

- **Absolute Speedup:** Calculated relative to the sequential execution time (532.55 s), it measures the real-world gain of the entire parallel architecture.
- **Relative Speedup and Efficiency:** Calculated relative to the MPI+OMP execution time on a single node (82.55 s), these metrics isolate the algorithm’s capacity to scale horizontally across the network, excluding the baseline benefits of local parallelization.

The global results are reported in Table 7, the graphical trends in Figure 2, and the phase-by-phase breakdown in Figure 3.

Num. Nodes	Time (sec)	Abs. Speedup	Rel. Speedup	Rel. Efficiency
1	82.55	6.45	1.00	100.0%
2	45.67	11.66	1.81	90.4%
4	33.90	15.71	2.44	60.9%
8	29.21	18.23	2.83	35.3%

Table 7: Strong Scalability results. The Absolute Speedup uses a reference sequential time of 532.55 s.

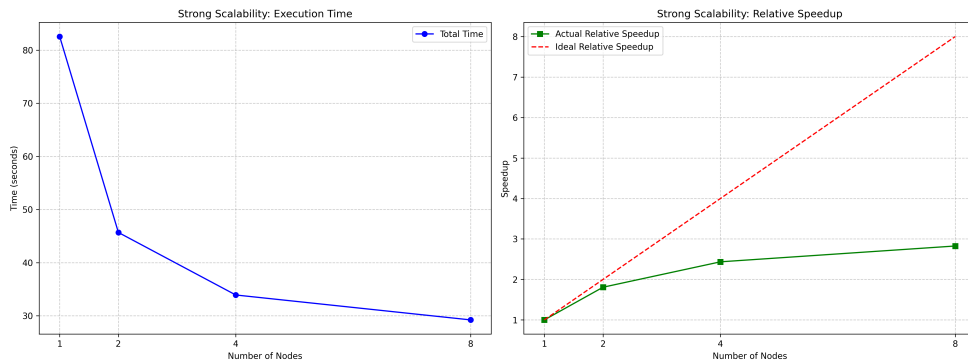


Figure 2: Total execution time and Relative Speedup as the number of physical nodes increases.

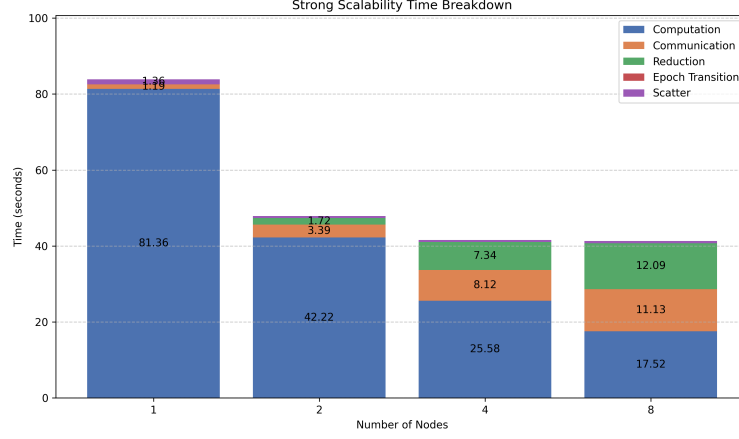


Figure 3: Per-rank peak time of each phase as the number of nodes increases (see methodological note in the text).

The decoupling of pure computation from network layers reveals clear architectural dynamics. First, the Computation peak scales strongly with node count, dropping from 81.36 s (1 node) to 42.22 s (2 nodes) to 17.52 s (8 nodes), confirming that the NNZ-based physical partitioning distributes the workload effectively. Since N is fixed, the divergence from ideal scaling at larger scales stems from two factors: as the per-rank workload shrinks, the reported peak (the maximum time across ranks) becomes sensitive to residual imbalances in NNZ partitioning, while fixed per-iteration bookkeeping overheads weigh proportionally more.

Conversely, network wait times grow explosively, with the Communication+Reduction peak rising from ≈ 1.2 s to ≈ 23.2 s (8 nodes). This spike reflects longer synchronization delays for faster ranks, caused by the inherent payload imbalance of NNZ row partitioning during `MPI_Allgatherv`. Consequently, while the absolute best execution time is achieved at 8 nodes (29.21 s), marginal gains drop rapidly beyond 2–4 nodes. Relative efficiency falls from 90.4% (2 nodes) to 35.3% (8 nodes), as growing collective communication overheads absorb the theoretical benefits of additional nodes.

4.6 Weak Scalability Analysis

To complete the performance investigation, *Weak Scalability* was evaluated. Unlike Strong Scalability, here the workload assigned to each node stays constant while both the problem size and the number of resources grow in parallel.

Since the workload of an SpMV kernel depends on both N (rows/columns) and NZ (non-zeros), the experiment was split into two configurations:

1. **Proportional Growth:** the average number of non-zeros per row stays constant; as nodes increase, both the matrix and the global vector x grow.
2. **Constant Vector Size (N fixed, NZ scales):** N is fixed at 1000000; doubling the nodes only doubles NZ . This isolates the network’s impact (the exchanged vector stays the same size) while progressively densifying the matrix.

4.6.1 Results: Proportional Growth

Starting from a base case of $N = 125,000$ and $NZ = 31,250,000$ per node, the resulting behavior is shown in Table 8. Figures 4 and 5 show the time/efficiency trend and the per-phase breakdown, respectively.

Nodes	Global N	Global NZ	Total Time (s)	Weak-Scaling Efficiency (E_w)
1	125,000	31,250,000	10.02	100.0%
2	250,000	62,500,000	15.95	62.8%
4	500,000	125,000,000	19.68	50.9%
8	1,000,000	250,000,000	28.84	34.7%

Table 8: Weak Scalability with proportional growth of rows and non-zeros.

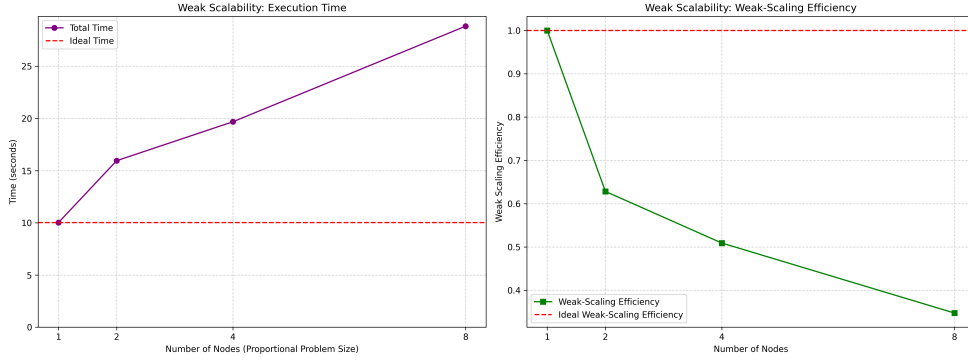


Figure 4: Total execution time and Weak-Scaling Efficiency, proportional growth of N and NZ .

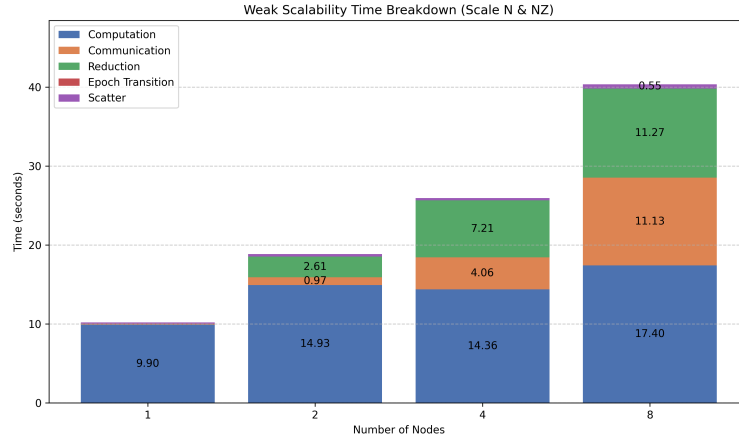


Figure 5: Per-rank peak of each phase, proportional growth of N and NZ .

Weak-Scaling Efficiency degrades quickly (34.7% at 8 nodes). Looking at the per-rank peak of the Computation component (Figure 5), which rises from 9.90 s (1 node) to 17.40 s (8 nodes) despite the per-node NZ load staying fixed ($\approx 31.25M$), a second factor emerges beyond the network: growing N means column indices are drawn from an increasingly larger global vector x . At 8 nodes ($N = 10^6$), the growth in the vector’s working set likely exceeds local L2/L3 cache capacity, suggesting an increase in cache misses that degrades pure computational efficiency, a Memory Wall effect distinct from, and additional to, the network cost.

4.6.2 Results: Constant Vector Size Scaling

To decouple pure computational scaling from vector-related network congestion, the experiment was repeated fixing $N = 1\,000\,000$ for every run, reaching the original target size at 8 nodes ($NZ = 250\,000\,000$). Results are in Table 9, Figures 6 and 7.

Nodes	Global NZ	Total Time (s)	Weak-Scaling Efficiency (E_w)
1	31,250,000	13.05	100.0%
2	62,500,000	14.43	90.4%
4	125,000,000	20.75	62.9%
8	250,000,000	29.40	44.4%

Table 9: Weak Scalability keeping N constant and densifying non-zeros.

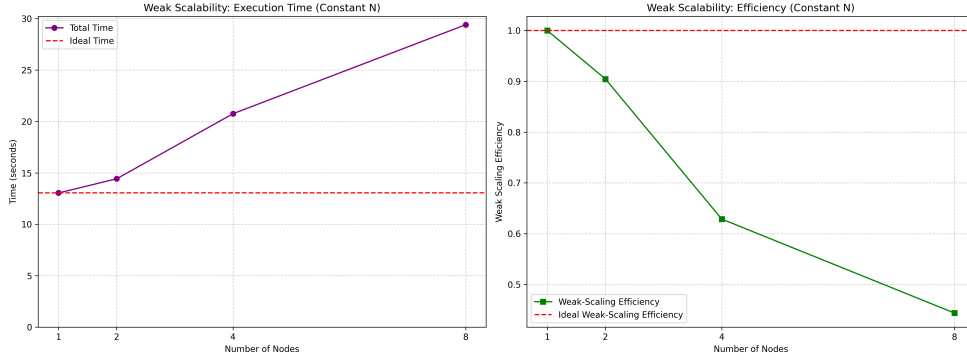


Figure 6: Total execution time and Weak-Scaling Efficiency, constant N .

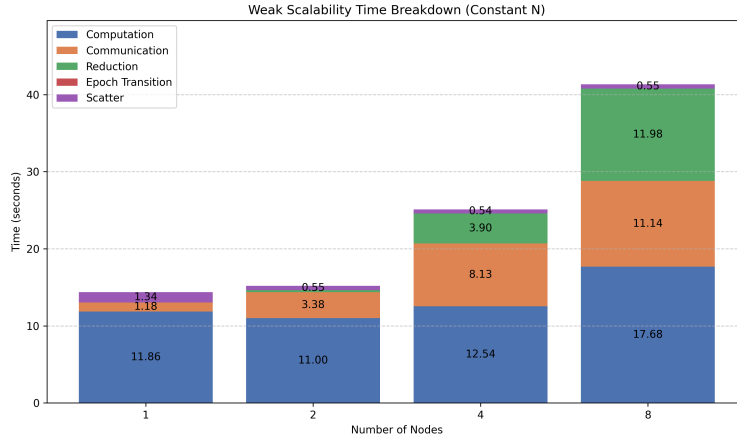


Figure 7: Per-rank peak of each phase, constant N .

With N fixed, the Computation peak stays stable: at 4 nodes it is nearly identical to the single-node case (12.54 s versus 11.86 s), confirming that, isolated from vector growth, local computation scales very well. The Weak-Scaling Efficiency drop (44.4% at 8 nodes) is therefore almost entirely attributable to the network: the Communication and Reduction peaks rise to 11.14 s and 11.98 s respectively at 8 nodes, up from a few tenths of a second at 1 node.

This conclusion is corroborated by two independent cross-checks. First, the Communication peak at 8 nodes (11.14 s) nearly exactly matches the one measured in the Strong Scalability experiment at the same node count/topology (11.13 s, Figure 3), which is expected since $N = 10^6$ is identical in both cases and the payload exchanged via `MPI_Allgatherv` depends only on that. Second, summing Communication and Reduction at 8 nodes gives an almost identical total between the two weak-scaling scenarios despite their equal NZ load at that scale (≈ 22.4 s in the proportional case versus ≈ 23.1 s in the constant- N case): the network "sees" only the global vector size, not the matrix density or local compute time.

In conclusion, Weak Scalability isolates two distinct, independent bottlenecks: a Memory Wall effect on the global vector, which only emerges when N grows with scale, and a network cost tied exclusively to N and topology, independent of computational load. The latter is the same mechanism already identified in the Strong Scalability analysis, here confirmed by a numerical cross-check between different experiments.

5 Task-Based vs Work-Sharing Scheduling

5.1 Work-Sharing OpenMP

This variant replaces dynamically generated tasks with OpenMP’s work-sharing constructs (`#pragma omp for`) within a single, persistent `#pragma omp parallel` region. Serial fragments (e.g., RNG initialization, `row_shift` updates, vector swapping, and timing) are isolated using `#pragma omp single` blocks, while computational kernels are executed collectively by the entire team. Load balancing is tailored to the workload: `spmv_csr_shifted_rows_omp_for` uses `schedule(dynamic, chunk_size)` to handle the irregular nonzero distribution of matrix rows, whereas vector operations (`dot_omp_for`, `scale_omp_for`) rely on `schedule(static, chunk_size)` for their uniform, predictable cost.

Synchronization relies entirely on the implicit barriers of `for` and `single` constructs, removing the need for explicit waits. Reductions use the standard `reduction(+:sum)` clause. To minimize synchronization overhead, the scale factor `inv` is redundantly computed by every thread rather than being calculated in a `single` block and broadcast. As a final observation, state variables that must persist across phase boundaries, such as the timer `t0`, are explicitly declared as `shared`, ensuring visibility since consecutive `single` blocks may be executed by different physical threads.

5.2 MPI + OpenMP Work-Sharing

This version merges the intra-node Work-Sharing strategy with the inter-node distribution of the MPI hybrid model. Crucially, the physical partitioning (`partition_rows_by_nnz`), gathering (`AllgatherPlan`), and shift-as-permutation logic remain identical to the MPI + OpenMP task-based version. The persistent parallel region is maintained per rank. Because `#pragma omp for` cannot be nested inside `#pragma omp single`, computation and MPI collectives are strictly separated along phase boundaries, unlike the interleaved structure of the task-based version. To comply with `MPI_THREAD_FUNNELED`, MPI collectives (`MPI_Allreduce`, `MPI_Allgatherv`) are explicitly confined to these `single` blocks.

The implicit barriers closing each `single` block seamlessly enforce execution ordering, ensuring all threads observe newly communicated data before the next collective `omp for` loop begins. The two-level reduction pattern is preserved: `local_dot_omp_for` performs a rank-local reduction via `reduction(+:sum)`, which is then globally combined using a single `MPI_Allreduce` call. Afterward, threads redundantly recompute the scaling factor locally. Scheduling strategies (dynamic for SpMV, static for vector kernels) remain identical to the pure OpenMP version.

5.3 Performance Evaluation: Task vs Work-Sharing

To evaluate the overhead of explicit task generation against implicit loop scheduling, the `#pragma omp task` model was compared with the `#pragma omp for` work-sharing construct. The analysis focused on the large irregular matrix ($N = 1\,000\,000$, $NZ = 250\,000\,000$), varying both intra-node thread concurrency and inter-node distribution scale.

Table 10 reports the evaluation executed on shared memory. Table 11 details the distributed MPI+OpenMP architecture, scaling from 1 to 8 nodes (16 threads per rank). The corresponding execution trends are illustrated in Figure 8.

Threads	Tasks Tot. (s)	Work-Share Tot. (s)	Tasks Comp. (s)	Work-Share Comp. (s)
4	276.81	279.72	276.80	279.71
8	148.48	149.43	148.47	149.43
16	80.30	79.91	80.29	79.90
32	49.43	48.44	49.43	48.42

Table 10: Single-node total and computation times comparing OpenMP Tasks vs Work-Sharing.

Nodes	Tasks Tot. (s)	Work-Share Tot. (s)	Tasks Comp. (s)	Work-Share Comp. (s)
1	82.63	81.04	81.44	79.85
2	45.74	45.78	42.18	42.29
4	34.01	33.90	25.86	25.59
8	26.31	28.35	15.64	16.89

Table 11: Total and local computation times comparing Task-based and Work-Sharing hybrid models.

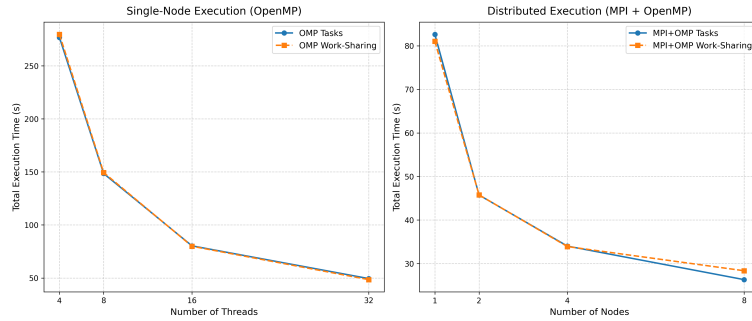


Figure 8: Performance comparison between Task-based and Work-Sharing models on Single-Node (left) and Distributed (right) architectures.

The empirical data reveals no systematic advantage for either paradigm at most scales tested, with a possible exception at the highest fragmentation level. On a single node, total-time differences between the task-based and work-sharing models remain within 0.5–2.0% across all thread counts (4 through 32), consistent with measurement noise rather than a genuine architectural effect. The same holds at low-to-moderate distributed scale (1 to 4 nodes), where differences stay within 0.1–1.9%. At 8 nodes, work-sharing takes approximately two seconds longer than the explicit task model (28.35 s vs 26.31 s, a +7.75% increase relative to the task-based time), a gap primarily driven by the local computation phase (16.89 s vs 15.64 s), though comparable to the run-to-run variation observed for this configuration elsewhere in this report, so it should be read as a mild trend rather than conclusive evidence. Architecturally, the decentralized work-stealing of task queues remains a plausible explanation, as it can absorb per-rank load imbalance more flexibly than the implicit barriers closing each `omp for` loop.

6 Conclusions

This project evaluated the performance of an iterative sparse matrix-vector computation on evolving matrices across shared-memory and distributed-memory environments, focusing on the architectural impact of irregular sparsity patterns.

The shared-memory parallelization strategies (C++ Thread Pool and OpenMP tasks) successfully mitigated the extreme workload imbalance of the irregular matrix. By leveraging dynamic work distribution, via concurrent task queues and OpenMP work-stealing, they seamlessly reassigned computations to idle threads, restricting performance fluctuations to a negligible margin compared to the ideal regular workload.

The hybrid MPI+OpenMP implementation achieved the absolute fastest execution times, but its strong and weak scalability degraded significantly when transitioning to the irregular workload (+67.6% execution time). The root cause of this limitation is not computational, but rather a structural communication bottleneck.

To ensure equal arithmetic workloads across nodes, the matrix rows were partitioned based on their cumulative number of non-zero elements. While this perfectly balances the local computation, it intrinsically assigns vastly different row counts to each MPI process. Consequently, during the `MPI_Allgatherv` synchronization phase, processes assigned to dense regions transmit tiny vector chunks, whereas processes handling sparse regions must transmit massive payloads over the network. As the node count increases and local computation accelerates, this extreme payload imbalance causes faster ranks to idle at synchronization barriers, making network communication the primary scalability bottleneck. Additionally, weak scalability tests isolated a secondary *Memory Wall* effect: when the global vector scales to $N = 10^6$, the working set exceeds local cache capacities, introducing severe cache-miss penalties completely independent of the network overhead.

Finally, the comparison between task-based and work-sharing scheduling revealed that neither model is universally superior. At most scales tested, differences between the two paradigms stayed within measurement noise; only at the highest fragmentation level (8 distributed nodes) did task-based scheduling show a mild edge, though comparable to the run-to-run variation observed elsewhere, preventing any definitive generalization.